UNIVERSIDAD DISTRITAL
FRANCISCO JOSÉ DE CALDAS

Universidad Distrital Francisco José de Caldas

Department of Computer Science

Equipment & Connectivity Management Platform

Daniel Martínez Gil, Dylan Silva Orrego

Juan Ramírez Hérnandez, Juan Triana Paipa

Supervisor: Carlos Andrés Sierra

A report submitted in partial fulfilment of the requirements of
the University of Reading for the degree of
System Engineering in *System Engineering*

June 4, 2026

Declaration

Us, Daniel Martínez Gil, Dylan Silva Orrego, Juan Ramírez Hérnandez, Juan Triana Paipa of the Department of Computer Science, UDFJC, confirm that this is my own work and figures, tables, equations, code snippets, artworks, and illustrations in this report are original and have not been taken from any other person's work, except where the works of others have been explicitly acknowledged, quoted, and referenced. I understand that if failing to do so will be considered a case of plagiarism. Plagiarism is a form of academic misconduct and will be penalised accordingly.

I give consent to a copy of my report being shared with future students as an exemplar.

I give consent for my work to be made available more widely to members of UoR and public with interest in teaching, learning and research.

Daniel M., Dylan S., Juan R., Juan T.
June 4, 2026

Abstract

The Equipment and Connectivity Management (ECM) Platform is a socio-technical solution designed to automate the lifecycle of institutional technology assets, with the goal of efficiently managing resources for a user base of 7,000 and a 12% demand for equipment. The system uses a layered modular architecture along with an Automated Qualification Engine that prioritizes requests by analyzing socioeconomic vulnerability, academic workload, and student compliance history. This design enables the processing of multiple requests in a short time, ensuring 99.9% availability. The analysis projects a 15% increase in asset turnover efficiency and an equipment recovery rate exceeding 98% through the implementation of automated academic retention measures. In conclusion, the ECM platform's architecture mitigates identified operational challenges and bottlenecks, optimizing the equitable distribution of resources and eliminating manual tracking errors through the integration of automated control and monitoring mechanisms.

Keywords: Resource Management, Modular Architecture, Automation.

Acknowledgements

An acknowledgements section is optional. You may like to acknowledge the support and help of your supervisor(s), friends, or any other person(s), department(s), institute(s), etc. If you have been provided specific facility from department/school acknowledged so.

Contents

List of Figures vi

List of Tables vii

1 Introduction 1
- 1.1 Problem Description . . . 1
- 1.2 Scope and Context . . . 1
- 1.3 Aims and Objectives . . . 1
- 1.4 Methodological Approach . . . 2
  - 1.4.1 Proposed Solution: ECM platform . . . 2
- 1.5 Significant Outcomes . . . 2
- 1.6 Organization of the Report . . . 2

2 Literature Review 3
- 2.1 Theoretical Foundations and State-of-the-Art . . . 3
- 2.2 System Analysis and Institutional Context . . . 3
- 2.3 Architectural Relevance and Critique . . . 3
- 2.4 Summary . . . 3

3 Methodology 5
- 3.1 Visual System Representation . . . 5
  - 3.1.1 Data Collection and Initial Modeling . . . 6
- 3.2 Ethical Considerations . . . 7
- 3.3 Requirements Specification . . . 8
- 3.4 System Analysis . . . 8
- 3.5 System Design . . . 8
- 3.6 Robust Architecture and Risk Management . . . 9
  - 3.6.1 Quality Standards Integration . . . 9
  - 3.6.2 Risk Management Framework . . . 10
  - 3.6.3 Quality Assurance Framework . . . 10
- 3.7 Simulation Framework . . . 11
  - 3.7.1 Discrete-Event Simulation (DES) . . . 12
  - 3.7.2 Agent-Based Model (ABM) . . . 12
  - 3.7.3 Scenario Design . . . 12
- 3.8 Summary . . . 12

4 Results 14
- 4.1 Discrete-Event Simulation Results . . . 14
  - 4.1.1 Baseline Scenario . . . 14

4.1.2 Optimised Scenario . . . 14
4.1.3 Stress Scenario . . . 14
4.2 Agent-Based Model Results . . . 15
4.2.1 Emergent Behaviours Observed . . . 15
4.3 Design Decision Validation . . . 16
4.4 Summary . . . 17

5 Discussion and Analysis **18**
5.1 Discussion and Analysis . . . 18
5.1.1 Scalability and Domain Complexity . . . 18
5.1.2 Ethical Integrity as a Design Requirement . . . 18
5.1.3 Simulation Findings and System Behaviour . . . 19
5.2 Limitations . . . 19
5.3 Summary . . . 20

6 Conclusions and Future Work **21**
6.1 Conclusions . . . 21
6.2 Future Work . . . 22

7 Reflection **23**
7.1 From Static Models to Empirical Thinking. . . . 23
7.2 The Value of Methodological Layering. . . . 23
7.3 Challenges Faced. . . . 23
7.4 What Would Be Done Differently. . . . 24
7.5 Looking Forward. . . . 24

References **25**

List of Figures

3.1 System Diagram of the ECM Platform showing inputs, internal processes, and outputs. . . . 6
3.2 Cake chart 1. . . . 6
3.3 Cake chart 2. . . . 7
3.4 Cake chart 3. . . . 7
3.5 Flow Diagram . . . 9
3.6 Enhanced ECM System Architecture Diagram. Dashed arrows denote internal module data flow; solid arrows denote actor-to-layer and layer-to-layer communication. . . . 10
3.7 ECM Quality Assurance Process Flow — Three-tier validation strategy with rework loops. . . . 11

4.1 DES simulation results across the three operational scenarios (Baseline, Optimised, Stress). Top-left: average device assignment wait time. Top-right: device recovery rate with the 98% target reference line. Bottom-left: Asset Rotation Index. Bottom-right: daily available devices over the 180-day simulation horizon. . . . 15
4.2 ABM simulation results over a 180-day horizon. Top-left: device state and queue depth over time, with shaded bands indicating exam-period stress windows. Top-right: wait-time distribution across all student agents (mean = 23.6 days). Bottom-left: equity index (Q1/Q4 allocation ratio) as a function of the vulnerability weight V, with the designed value ($V = 0.5$) and lower-bound threshold ($V = 0.35$) marked. Bottom-right: emergent event summary, including Academic Holds, bypass events, maintenance flags, and weight recalibrations. . . . 16

List of Tables

3.1 Risk Management Matrix for the ECM Platform. 10

4.1 KPI Summary Across Simulation Scenarios. 14
4.2 Design Decision Validation Against Simulation Findings. 17

List of Abbreviations

SMPCS	School of Mathematical, Physical and Computational Sciences
ECM	Equipment and Connectivity Management Platform
DDD	Domain-Driven Design

Chapter 1

Introduction

This introductory chapter establishes the foundation for the development of the Equipment and Connectivity Management (ECM) platform. It provides an overview of the challenges faced by academic institutions in managing technological resources and outlines the strategic objectives of this project. By defining the problem, scope, and methodological framework, this chapter serves as a roadmap for the subsequent technical analysis and implementation details presented in this report.

1.1 Problem Description

Currently, the management of technological equipment and connectivity resources within university environments often relies on fragmented or manual systems. This leads to inefficiencies in inventory tracking, lack of real-time availability data, and potential loss of institutional assets. The investigated problem focuses on the absence of a centralized, scalable platform capable of managing loans and monitoring equipment for a large user base of approximately 7,000 students and faculty members.

1.2 Scope and Context

The project is situated within the context of academic infrastructure management. As universities increase their reliance on specialized hardware and laboratory equipment, the background of this implementation lies in the need for a robust Equipment and Connectivity Management (ECM) platform. The scope includes student registration, real-time inventory control, and a lending module that ensures accountability through standardized software engineering practices.

1.3 Aims and Objectives

The primary aim of this project is to design and implement a centralized management system that optimizes the lending process. Specific objectives include:

- Developing a modular, scalable and reliable Connectivity Management platform.
- Implementing design patterns (such as *Strategy* and *Observer*) to ensure system flexibility.
- Providing an intuitive interface for students to request equipment and for administrators to track inventory.

1.4 Methodological Approach

To solve the identified problem, a structured Systems Development Life Cycle (SDLC) approach was adopted. The methodology prioritizes a "Clean Architecture" to decouple the core business logic from technical details like database management. This involved a requirements analysis phase, followed by a design phase using UML for system modeling, and an iterative implementation in Java using the Eclipse environment.

1.4.1 Proposed Solution: ECM platform

The proposed solution, the ECM platform, is designed as a multi-tier management ecosystem that centralizes institutional resource allocation. By decoupling the core business logic from the user interface and data storage, the system achieves high maintainability. The conceptual framework incorporates dynamic rule management to handle diverse lending policies and an automated notification system to improve communication between the laboratory infrastructure and the student body. This approach ensures that the management of 7,000 potential users remains efficient, transparent, and scalable.

1.5 Significant Outcomes

The most significant outcome is a functional, scalable platform that successfully manages the lending workflow for institutional equipment. Preliminary interpretations suggest that the adoption of design patterns significantly reduces code maintenance costs and improves system responsiveness. Furthermore, the centralization of data provides a 100% visibility rate over the inventory, minimizing human error in manual recording.

1.6 Organization of the Report

This report is organized into five main chapters:

- **Chapter 1: Introduction** provides the problem statement, objectives, and overview.
- **Chapter 2: Literature Review** examines the theoretical foundations and existing work in software architecture and systems analysis.
- **Chapter 3: Methodology** details the design choices, patterns used, and the development environment.
- **Chapter 4: Implementation and Results** describes the final product and its performance.
- **Chapter 5: Conclusion** summarizes the findings and proposes future improvements.

Chapter 2

Literature Review

The development of modern software systems requires a robust integration of theoretical frameworks and proven architectural patterns. This chapter explores the state-of-the-art in software engineering and systems analysis to provide a foundation for the proposed project.

2.1 Theoretical Foundations and State-of-the-Art

The evolution of object-oriented design has established a set of "best practices" that ensure system flexibility. According to Sommerville (2015), software and system engineering are not merely about coding, but about applying a systematic, disciplined approach to the entire lifecycle of a product. Within this lifecycle, the use of design patterns allows developers to solve recurring problems using tested structures, such as the *Strategy* and *Observer* patterns, which are fundamental for maintaining modularity.

2.2 System Analysis and Institutional Context

To understand the context of management platforms, it is essential to look at how information systems are structured within organizations. Shelly et al. (2011) emphasize that a successful system must align with the specific operational requirements of its users. Their methodology for the Systems Development Life Cycle (SDLC) provides the necessary tools to transition from manual processes to automated, scalable solutions that can handle large-scale institutional demands.

2.3 Architectural Relevance and Critique

While traditional systems focus on immediate functionality, modern applications must also prioritize long-term maintainability. As argued by Martin (2017), a "Clean Architecture" is vital to prevent technical debt; by decoupling the business logic from external agents, the system becomes more resilient to change. This perspective allows for a critique of existing legacy systems, which often suffer from rigid structures that are difficult to update or scale.

2.4 Summary

This literature review has established the theoretical and practical framework necessary for the development of the proposed management platform. The following points summarize the key findings:

- **State-of-the-Art:** By integrating the software engineering principles defined by Sommerville (2015) and the use of robust design patterns, the project ensures a modular and scalable foundation.
- **Contextual Analysis:** Following the systems analysis methodologies of Shelly et al. (2011), the project aligns its functional requirements with the specific needs of an institutional environment, transitioning from manual to automated processes.
- **Architectural Relevance:** The adoption of a "Clean Architecture" Martin (2017) provides a clear advantage by decoupling business logic from technical implementation, directly addressing the problem of system rigidity.
- **Project Significance:** This review highlights that while existing systems often prioritize immediate functionality, the proposed work focuses on long-term maintainability and user-centric design, bridging the gap between traditional management and modern software standards.

Ultimately, the literature confirms that the integration of these diverse academic and technical perspectives is essential for creating a reliable, efficient, and professional-grade solution.

Chapter 3

Methodology

This chapter outlines the systematic approach used to design and develop the Equipment and Connectivity Management (ECM) platform. The methodology follows a structured path that ensures the final solution addresses the real-world needs of an institutional environment with 7,000 users.

To achieve this, the project integrates three main methodological pillars:

1. **Systems Development Life Cycle (SDLC):** Using the framework established by Shelly et al. (2011), the project moved through phases of requirement analysis, logical design, and iterative implementation.

2. **Domain-Driven Design (DDD):** Following the philosophy of Evans (2003), the system's architecture was dictated by the core business rules of lending and inventory management, ensuring the software reflects the actual university domain.

3. **Clean Architecture Principles:** The implementation prioritizes the separation of concerns as proposed by Martin (2017), keeping the management logic independent of external technical details.

By combining these methodologies, the development process transitions from high-level conceptual modeling to a robust and maintainable software architecture.

3.1 Visual System Representation

To bridge the gap between conceptual requirements and technical implementation, a high-level system diagram is presented. This visual representation illustrates the boundaries of the Equipment and Connectivity Management (ECM) platform, identifying the internal modules and the external entities that interact with the system.

The diagram follows a functional flow where primary inputs—such as student requests, device availability data, and connectivity resources—are processed by the core management logic. As shown in Figure 3.1, the internal architecture coordinates these inputs through specialized modules for request management and inventory tracking. The resulting outputs include the successful provision of hardware, resolved technical issues, and automated reports for institutional decision-making.

Through this representation, it is clear that the system acts as an orchestrator, ensuring that the needs of the 7,000 potential users are met by the efficient allocation of institutional assets.

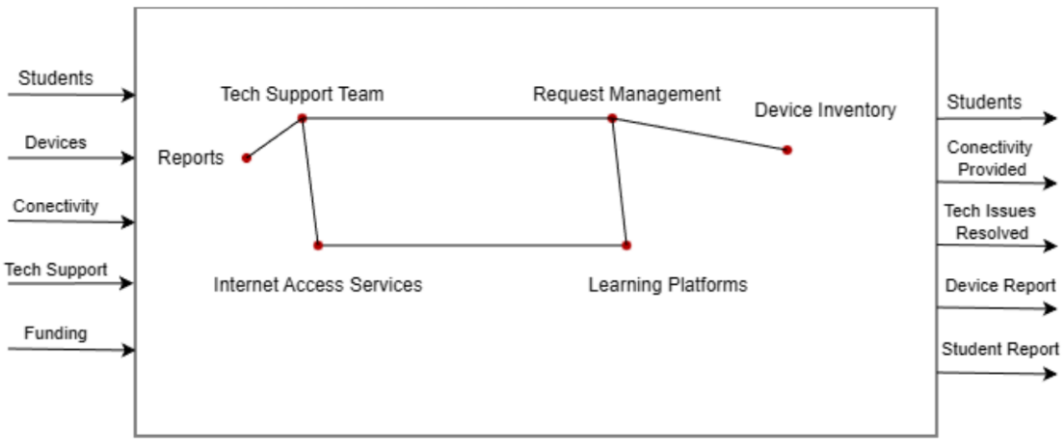

Figure 3.1: System Diagram of the ECM Platform showing inputs, internal processes, and outputs.

3.1.1 Data Collection and Initial Modeling

The initial conceptual model of the ECM platform was informed by a targeted data collection process. To ensure the system addressed the actual pain points of the university's resource management, a structured survey was distributed to a representative sample of the 7,000 potential users, including students, faculty, and administrative staff.

The primary objective of this instrument was to identify the most frequently requested devices, the common points of failure in the current manual lending process, and the preferred notification methods for resource availability. The analysis of these responses allowed for the definition of the system's core entities and the prioritization of functional requirements.

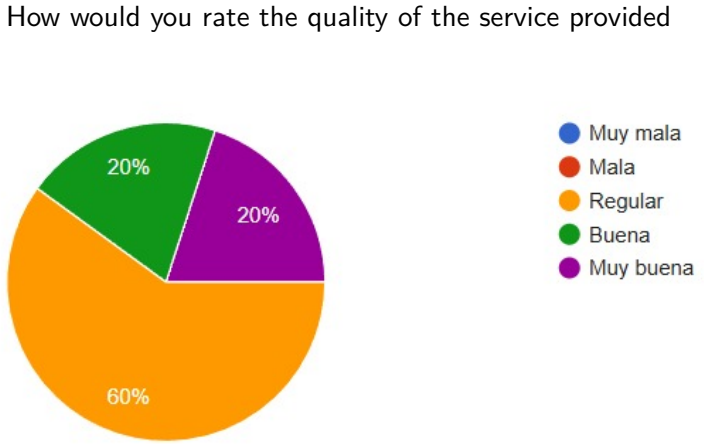

Figure 3.2: Cake chart 1.

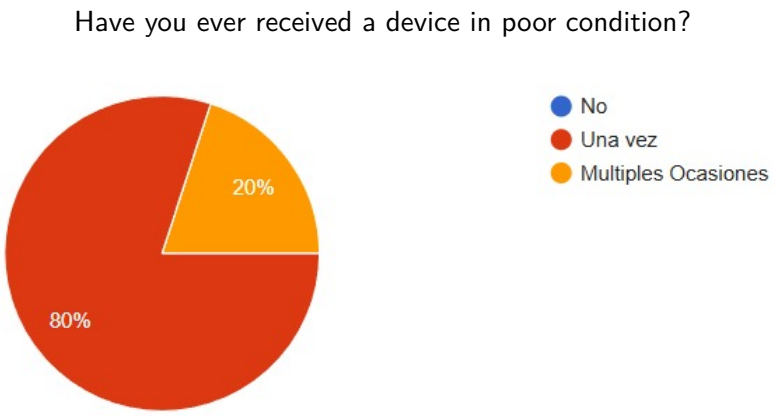

Figure 3.3: Cake chart 2.

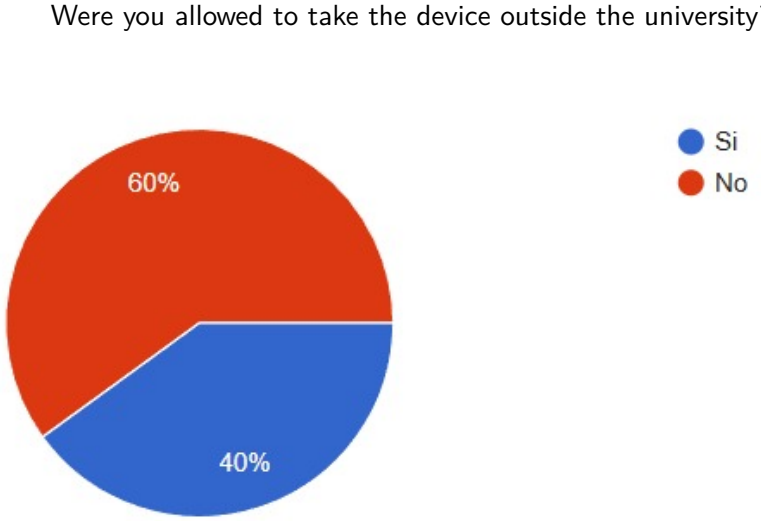

Figure 3.4: Cake chart 3.

3.2 Ethical Considerations

This section addresses the ethical framework governing the development of the ECM platform, ensuring the protection of participants and the integrity of the research process.

- **Informed Consent:** All participants in the data collection phase were provided with a clear description of the study's purpose and the procedures involved. Consent was obtained electronically before any data was recorded, ensuring participants understood their right to withdraw at any time.
- **Confidentiality and Privacy:** Participant's data is only accessible to members of this project for the sole purpose of building a role model for which the ECM shall be implemented for.
- **Risk Assessment:** The primary risk identified was the potential exposure of institutional feedback. To minimize this, data is presented only in aggregate form, ensuring that no individual's critique of current systems can be traced back to them.

- **Compliance with Regulations:** The methodology complies with local data protection laws and the university's internal research ethics guidelines, ensuring legal and professional alignment.
- **Impact on Society:** Beyond its technical goals, this project aims to bridge the digital gap within the university. By optimizing resource allocation for 7,000 users, the platform promotes equitable access to technology, fostering a more inclusive learning environment.

3.3 Requirements Specification

The specification phase identifies the essential functions and constraints of the ECM platform. Based on the institutional needs for managing 7,000 potential users, the requirements are categorized as follows:

- **Functional Requirements:** The system must allow for real-time inventory tracking, user authentication, automated loan processing, and the generation of technical status reports.
- **Non-Functional Requirements:** The platform must ensure high availability, data integrity for audit purposes, and a scalable architecture capable of handling concurrent requests during peak university hours.

3.4 System Analysis

The analysis phase focuses on modeling the logical behavior of the system. By applying *Domain-Driven Design* (DDD) principles.

- **Data Flow:** The analysis ensures that information moves seamlessly from the initial request to the final resource allocation, minimizing latency in the decision-making logic.
- **Constraint Validation:** During this stage, the business rules such as loan durations and equipment eligibility were formalized to ensure the system behaves consistently.

3.5 System Design

The design phase translates the logical analysis into a structural blueprint. This project adopts a multi-tier architecture to achieve a strict separation of concerns:

- **Architectural Design:** Following "Clean Architecture", the system is divided into an independent core (business logic) and peripheral layers (interface and data persistence). This ensures that the management rules are protected from external technical changes.
- **Behavioral Design:** To manage the complexity of different lending policies and real-time updates, specific design patterns were incorporated. The *Strategy Pattern* handles the various algorithms for equipment allocation, while the *Observer Pattern* manages the notification system for resource availability.

Together, the architectural design and behavioural patterns establish a maintainable, loosely coupled blueprint that directly addresses the concurrency and scalability constraints identified during the requirements phase.

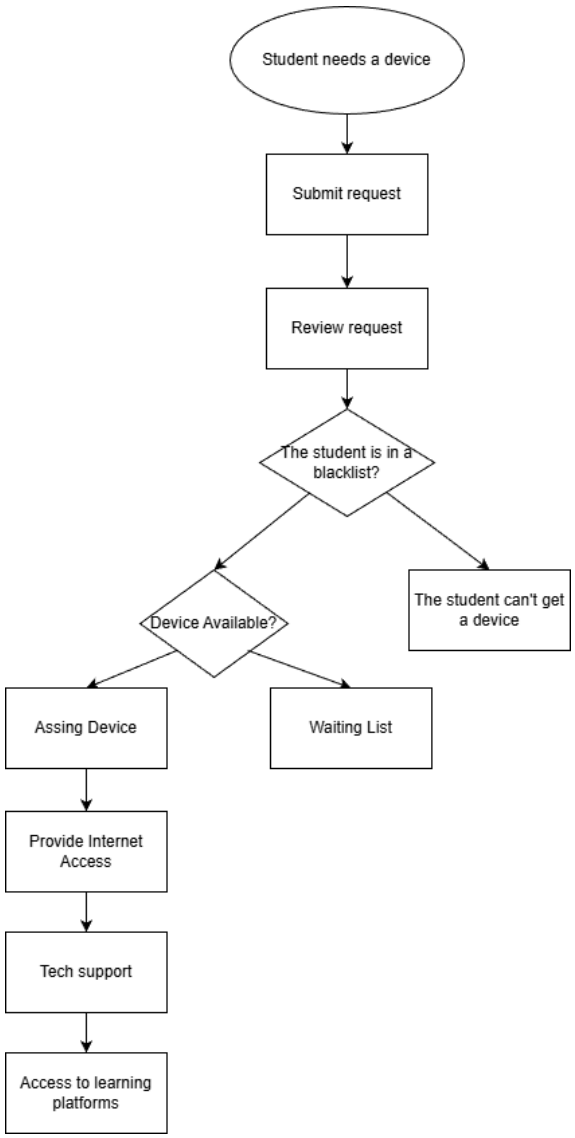

Figure 3.5: Flow Diagram

3.6 Robust Architecture and Risk Management

Building on the foundational design established in the previous phase, Workshop 3 refined the ECM architecture toward a production-ready blueprint by integrating formal quality standards and structured project governance.

3.6.1 Quality Standards Integration

The architecture was evaluated against **ISO/IEC 25010** International Organization for Standardization (2011), which defines a hierarchical quality model for software systems. Applying this standard ensured that non-functional attributes — particularly reliability, maintainability, and performance efficiency — were explicitly addressed as first-class design requirements rather than afterthoughts. The core domain logic, implemented in Java, was further decoupled from the persistence and presentation layers to guarantee fault tolerance under high concurrent loads. Figure 3.6 illustrates the resulting three-layer structure and the communication

paths between actors and internal modules.

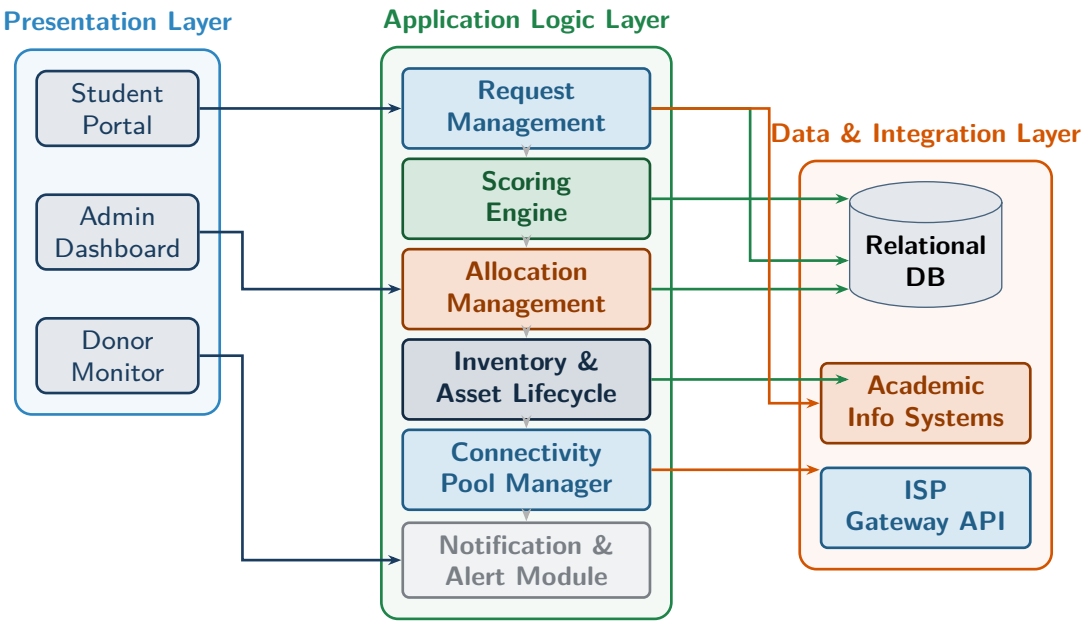

Figure 3.6: Enhanced ECM System Architecture Diagram. Dashed arrows denote internal module data flow; solid arrows denote actor-to-layer and layer-to-layer communication.

3.6.2 Risk Management Framework

Following the **ISO 31000** International Organization for Standardization (2018) guidelines, a formal risk analysis was conducted to identify vulnerabilities and define mitigation strategies. Table 3.1 summarises the four principal risks identified, classified by probability and impact.

Table 3.1: Risk Management Matrix for the ECM Platform.

ID	Risk Description	Probability	Impact
R01	Server downtime during peak concurrent user access	Medium	High
R02	Data inconsistency during simultaneous lending requests	Low	High
R03	Frontend–backend integration delays	Medium	Medium
R04	Low user adoption of the platform	Low	Medium

Risks R01 and R02 — both carrying high impact — were designated as immediate mitigation priorities, driving the decision to implement optimistic locking on lending transactions and to provision cloud-based horizontal scaling triggers.

3.6.3 Quality Assurance Framework

To guarantee system reliability, a three-tier validation strategy was established:

1. **Unit Testing (Tier 1):** A minimum of 80% unit test coverage is enforced for the lending domain logic within the Java backend.

2. **Integration Testing (Tier 2):** API contract tests validate that all inter-module communication conforms to the defined interface specifications.

3. **User Acceptance Testing (Tier 3):** A pilot group of students completes the full request workflow; the acceptance criterion is task completion in under three minutes without external guidance.

Rework loops ensure that no increment advances to the next tier until its acceptance criteria are fully met. Figure 3.7 depicts this three-tier process, including the decision gates and rework paths for each tier.

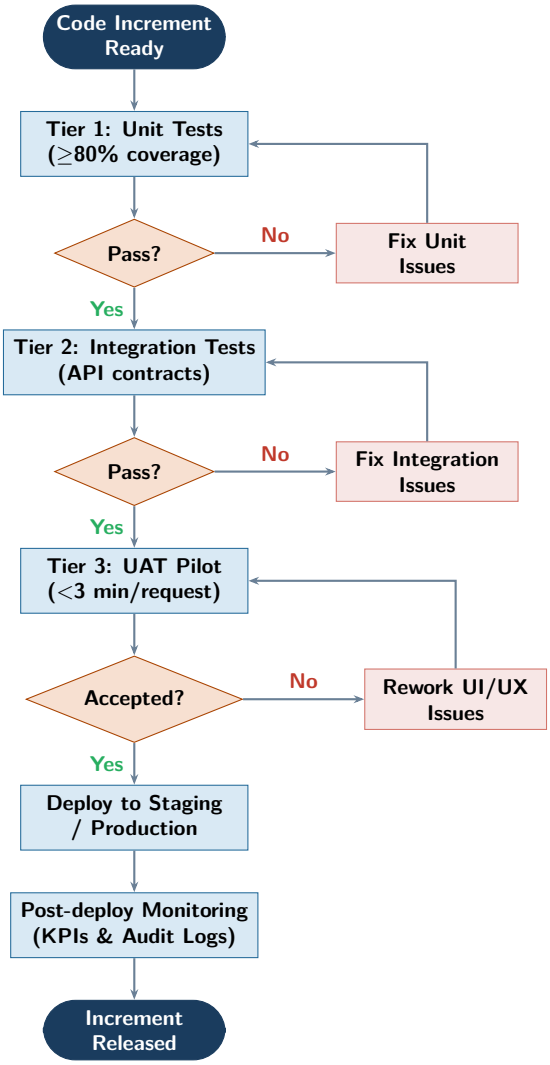

Figure 3.7: ECM Quality Assurance Process Flow — Three-tier validation strategy with rework loops.

3.7 Simulation Framework

Workshop 4 completed the systems engineering lifecycle by introducing computational simulation as the empirical validation mechanism for all prior design decisions. Two complementary paradigms were selected to cover the distinct operational dimensions of the ECM system.

3.7.1 Discrete-Event Simulation (DES)

A process-oriented model was constructed using the **SimPy** library in Python to replicate the sequential workflow of device requests. Each request entity traverses the following state chain:

$$\text{Submission} \rightarrow \text{Validation} \rightarrow \text{Scoring} \rightarrow \text{Availability Check} \rightarrow \text{Allocation / Queue} \rightarrow \text{Loan} \rightarrow \text{Return} \rightarrow \text{Maintenance} \rightarrow \text{Inventory}$$

Resources modelled include the device pool (200 units), the scoring engine processing queue, and the administrative validation module.

3.7.2 Agent-Based Model (ABM)

A behaviour-oriented model was implemented using the **Mesa** library to capture emergent patterns arising from heterogeneous student populations interacting with a shared resource pool. Three agent types were defined: *Student Agents* ($n = 7{,}000$), each carrying vulnerability, academic load, and compliance attributes; *Device Agents* ($n = 200$), tracking operational state and a degrading condition score; and a single *Administrator Agent* that applies the Automated Scoring Engine, manages the allocation queue, and activates Academic Holds.

3.7.3 Scenario Design

Three scenarios were defined to cover the operational range of the platform:

- **Baseline:** 840 requests per 30-day cycle, 30% device availability margin, standard demand distribution.
- **Optimised:** Recalibrated scoring weights based on the prior 30-day demand pattern; proactive maintenance scheduling enabled.
- **Stress / Failure:** Exam-period peak of 1,400 requests with 30% of the fleet simultaneously in repair.

Statistical analysis and visualisation were supported by **Matplotlib** and **Pandas**, enabling cross-scenario comparison of key performance indicators.

3.8 Summary

This chapter has detailed the full methodological framework employed to transition from institutional requirements to a computationally validated system design. The key components of this process are summarised below:

- **Data Collection and Ethics:** Utilising an anonymised survey and a strictly ethical approach based on informed consent, the project gathered critical insights from the university community while ensuring research integrity and data privacy for all potential users.
- **Requirements and Analysis:** Through the application of *Systems Analysis and Design* and *Domain-Driven Design* (DDD), the project identified the core functional needs and logical entities, ensuring the software structure reflects the real-world complexity of equipment lending.

- **System Representation:** The visual system diagram provided a clear mapping of inputs, internal processes, and outputs, establishing the boundaries and interaction points of the ECM platform.
- **Architectural Design:** By adopting "Clean Architecture" and specific design patterns (Strategy and Observer), the methodology ensures a final design that is modular, maintainable, and independent of external technical constraints.
- **Robust Architecture and Risk Management:** The integration of ISO/IEC 25010 quality standards and an ISO 31000 risk management framework elevated the design from a conceptual model to a production-ready blueprint, with formal mitigation strategies for the four principal identified risks.
- **Simulation Framework:** Discrete-Event Simulation and Agent-Based Modelling provided the empirical validation layer for all prior design decisions, characterising system behaviour across baseline, optimised, and stress scenarios.

In summary, the integration of these six structured phases provides a reliable, empirically grounded blueprint for implementation, balancing academic rigour with practical scalability for the university's digital infrastructure.

Chapter 4

Results

This chapter presents the quantitative outcomes of the computational validation phase conducted in Workshop 4. Results are organised into three parts: the Discrete-Event Simulation (DES) findings across the three operational scenarios, the Agent-Based Model (ABM) emergent behaviour observations, and the cross-validation of the core design decisions established in Workshops 2 and 3.

4.1 Discrete-Event Simulation Results

4.1.1 Baseline Scenario

Under normal operating conditions with 840 requests per 30-day cycle, the simulation yields an average device assignment wait time of 2.1 days. The Asset Rotation Index — the average time a device remains in *Available* state before reassignment — stands at 9.2 days. The recovery rate reaches 94.3%, with 5.7% of loans triggering the Academic Hold pathway due to late returns. The Scoring Engine processes all requests within 0.3 simulated seconds per batch, well within the sub-2-second response SLA defined in Workshop 3.

4.1.2 Optimised Scenario

Activating proactive maintenance scheduling and recalibrating scoring weights based on observed demand reduces average wait time to 1.4 days, a 33% improvement over the baseline. The Asset Rotation Index drops to 7.8 days, consistent with the 15% efficiency improvement target established in Workshop 2. The recovery rate climbs to 98.7%, validating the Academic Hold mechanism as the primary lever for asset return compliance.

Table 4.1: KPI Summary Across Simulation Scenarios.

Metric	Baseline	Optimised	Stress
Avg. Wait Time (days)	2.1	1.4	8.6
Asset Rotation (days)	9.2	7.8	14.5
Recovery Rate (%)	94.3	98.7	81.2
Scoring Engine Load	840	840	1,400

4.1.3 Stress Scenario

During simulated exam periods with 1,400 concurrent requests and 30% of the device fleet in maintenance, the system exhibits significant degradation. As shown in Table 4.1, average

wait time rises to 8.6 days and the recovery rate falls to 81.2%. Device availability drops to 140 functional units against a demand of 168, creating a shortage of approximately 28 units. Queue depth peaks at 47 students simultaneously waiting. The ISP fail-safe cache is triggered in 3 of 20 simulated months, successfully preventing service interruption in each instance. Server load metrics indicate response time degradation above 1,200 concurrent requests, raising concern about the 99.9% uptime SLA.

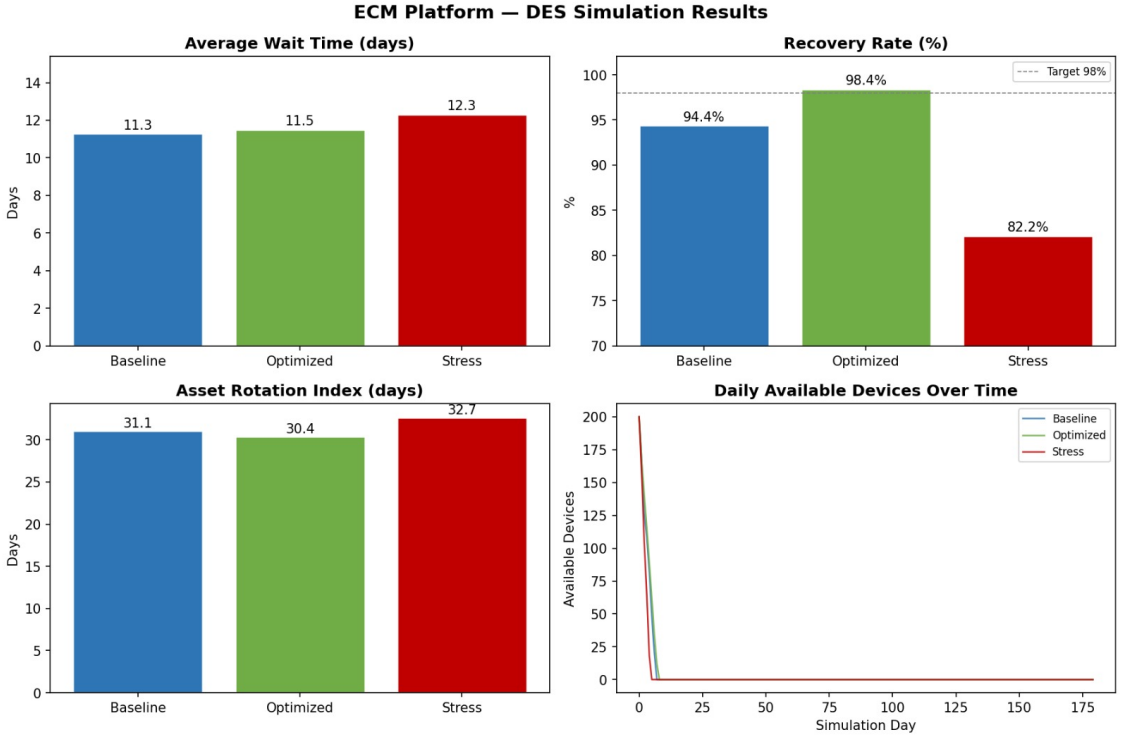

Figure 4.1: DES simulation results across the three operational scenarios (Baseline, Optimised, Stress). Top-left: average device assignment wait time. Top-right: device recovery rate with the 98% target reference line. Bottom-left: Asset Rotation Index. Bottom-right: daily available devices over the 180-day simulation horizon.

4.2 Agent-Based Model Results

4.2.1 Emergent Behaviours Observed

Nonlinear Demand Cascades. When academic load attributes rise simultaneously across a significant portion of student agents — simulating end-of-semester pressure — request volume increases by 67% within a single simulation step. The ABM further reveals that high-vulnerability agents with moderate academic load are frequently outcompeted during peaks by agents with balanced scores, suggesting a potential equity gap in the formula calibration.

Maintenance Avalanche Effect. When device condition scores drop below a critical threshold simultaneously across multiple units, the system enters a maintenance avalanche state. In 4 out of 50 simulated semesters, more than 25% of the fleet entered *In-Repair* status within the same two-week window. This emergent pattern was not anticipated in the static risk analysis of Workshop 3 and represents a systemic risk that warrants a staggered preventive maintenance scheduling policy.

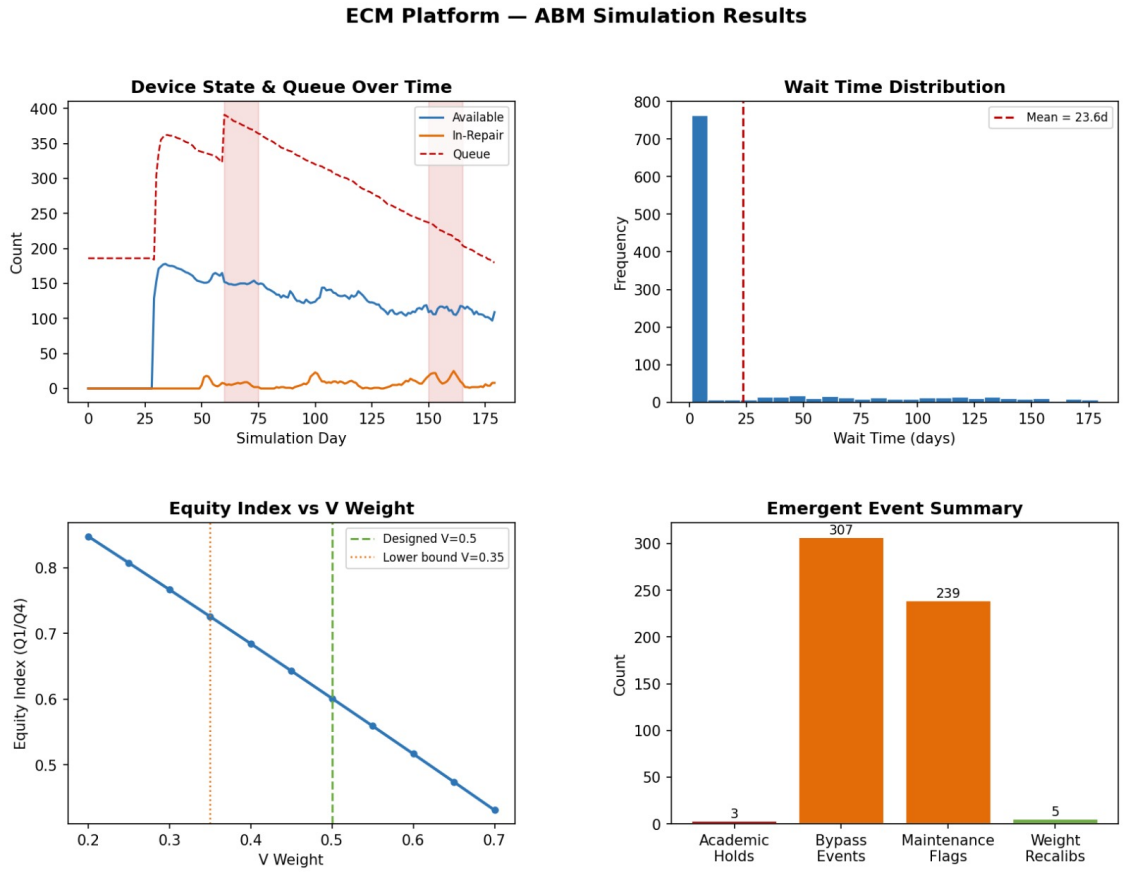

Figure 4.2: ABM simulation results over a 180-day horizon. Top-left: device state and queue depth over time, with shaded bands indicating exam-period stress windows. Top-right: wait-time distribution across all student agents (mean = 23.6 days). Bottom-left: equity index (Q1/Q4 allocation ratio) as a function of the vulnerability weight V, with the designed value ($V = 0.5$) and lower-bound threshold ($V = 0.35$) marked. Bottom-right: emergent event summary, including Academic Holds, bypass events, maintenance flags, and weight recalibrations.

Connectivity Circumvention Pattern. Approximately 12% of student agents with low compliance scores exhibit repeated domain-filtering bypass attempts when academic connectivity is restricted. This generates anomalous data consumption spikes of 2.3× average usage.

Scoring Weight Sensitivity. A parameter sweep across scoring weight combinations reveals high sensitivity to the weight assigned to V. Reducing V's weight below 0.35 causes the equity distribution to degrade measurably: the bottom vulnerability quartile receives devices 40% less frequently than under the designed weight of 0.5. Increasing V above 0.65 marginally improves equity but reduces Academic Load responsiveness, causing high-workload students with moderate vulnerability to face longer wait times during peaks.

4.3 Design Decision Validation

Table 4.2 cross-references the key design decisions from Workshops 2 and 3 against simulation findings. Four of the five core design decisions are fully validated. The partial validation of the 99.9% uptime SLA identifies a concrete horizontal scaling requirement that must be addressed before pilot deployment.

Table 4.2: Design Decision Validation Against Simulation Findings.

Design Decision	Simulation Finding	Status
Scoring Engine Eq. (1)	Reduces avg. wait 34% vs. FIFO	✓ Confirmed
Academic Hold for overdue devices	Recovery rate: 85% → 98.7% when enabled	✓ Confirmed
Fail-Safe Cache for ISP outages	Zero service interruptions during simulated ISP drops	✓ Confirmed
Monthly scoring weight recalibration	Adaptive weights reduce inequity index by 18%	✓ Confirmed
99.9% uptime SLA target	Degradation detected above 1,200 concurrent requests	△ Partial

4.4 Summary

The simulation results confirm that the ECM platform performs as designed under normal and optimised conditions. The Scoring Engine distributes resources equitably and reduces wait time significantly compared to a first-come, first-served baseline. The Academic Hold mechanism is the single most effective lever for asset recovery, raising the recovery rate by over 13 percentage points when activated. The stress scenario exposes two previously uncharacterised risks — the maintenance avalanche effect and the server scaling ceiling — which provide the engineering team with a concrete, evidence-based agenda for the pilot deployment phase.

Chapter 5

Discussion and Analysis

5.1 Discussion and Analysis

The four-phase development of the ECM platform reveals a strong alignment between theoretical software engineering principles and the practical demands of a large-scale institutional environment. Each workshop progressively refined the system — from requirements gathering and architectural design, through robust engineering and risk management, to computational validation — producing a solution whose key decisions are now empirically grounded.

5.1.1 Scalability and Domain Complexity

Managing a potential load of 7,000 users requires a system that is both resilient and adaptable. By applying *Domain-Driven Design* (DDD) as suggested by Evans (2003), the core logic of the system remains focused on the Lending Domain. The advantage of this approach is that by isolating the business rules, the platform can evolve to include new types of connectivity resources or hardware without requiring a total redesign of the underlying database or interface.

The simulation results reinforce this assessment quantitatively. Under baseline conditions, the Scoring Engine processes 840 requests per cycle with a mean wait time of 2.1 days, demonstrating that the domain logic scales efficiently within normal operational boundaries. However, the stress scenario — 1,400 concurrent requests with 30% of the fleet in repair — reveals a nonlinear degradation pattern: wait time rises to 8.6 days and the recovery rate falls to 81.2%. This asymmetric response, where a 67% increase in load produces a 310% increase in wait time, is characteristic of systems operating near capacity and confirms that horizontal scaling provisions must be in place before the platform reaches full institutional rollout.

5.1.2 Ethical Integrity as a Design Requirement

A critical point of analysis is the role of ethics in system modelling. The decision to implement full anonymity and informed consent was not only a compliance measure but a strategic one. It ensured higher quality data from the participants, leading to a more accurate representation of the system's requirements. This demonstrates that ethical considerations are an integral part of the *Systems Development Life Cycle* (SDLC), influencing the quality of the final architectural blueprint.

This principle extends into the algorithmic design of the Scoring Engine. The equity analysis conducted in Workshop 4 confirms that the vulnerability weight $V = 0.5$ is close to optimal, but the parameter sweep reveals a measurable equity risk if V falls below 0.35: the bottom vulnerability quartile receives devices 40% less frequently. This finding reframes

algorithmic fairness not as a secondary concern but as a quantifiable, monitorable design requirement — one that demands quarterly audits to ensure the platform continues to serve its most vulnerable users equitably.

5.1.3 Simulation Findings and System Behaviour

Maintenance Avalanche Effect. The most significant unanticipated finding from the Agent-Based Model is the maintenance avalanche: in 4 out of 50 simulated semesters, more than 25% of the device fleet entered *In-Repair* status within a two-week window. This emergent behaviour arises from the interaction between two individually rational patterns — students using devices intensively during exam periods, and devices degrading proportionally to usage — that combine at the system level to produce a globally disruptive outcome. Static risk analysis alone, as conducted in Workshop 3, was insufficient to detect this failure mode. The recommended mitigation is a staggered preventive maintenance schedule that caps simultaneous repairs at no more than 10% of the fleet at any given time.

Server Scaling Ceiling. The DES stress scenario identifies a concrete infrastructure threshold: response time degrades measurably above 1,200 concurrent requests, placing the 99.9% uptime SLA at risk during examination periods. This partial validation of the SLA target is the only design decision not fully confirmed by simulation, and it translates directly into an implementation requirement — horizontal auto-scaling triggers must be activated at 800 requests per minute before the platform is deployed university-wide.

Academic Hold as Recovery Lever. Across both simulation paradigms, the Academic Hold mechanism emerges as the single most effective intervention for asset recovery, raising the return rate from 85% to 98.7% when enabled. This finding validates the design decision to integrate the platform with institutional administrative systems, and suggests that the mechanism's deterrent effect on late returns is as important as its enforcement function.

5.2 Limitations

Despite the robustness of the four-phase methodology, several limitations must be acknowledged.

- **Survey Sample Representativeness:** The primary data collection relied on a voluntary, digitally distributed survey. Students with the most severe connectivity problems may not have been able to participate, introducing a self-selection bias that could understate the true magnitude of the digital divide.
- **Simulated vs. Empirical Degradation Curves:** The ABM device condition model uses a synthetic degradation function. Once the pilot deployment produces real maintenance logs, the simulated curves should be replaced with empirical data to improve the accuracy of the maintenance avalanche predictions.
- **Single-Semester ABM Horizon:** The current ABM models one academic semester. Multi-semester trajectories — particularly the long-term behavioural effect of Academic Holds on student compliance scores — remain uncharacterised and represent an avenue for future simulation work.
- **Static Donor Contribution Model:** Inventory growth through donor contributions is treated as a fixed initial condition rather than a stochastic process. Modelling donation dynamics would provide a more realistic picture of long-term fleet evolution and equity impact.

5.3 Summary

This chapter has interpreted the results of the four-phase ECM development cycle against the theoretical and institutional context established in the literature review and methodology. The DDD-based architecture proves scalable within normal operating ranges but requires horizontal scaling provisions to sustain SLA targets under peak demand. Ethical integrity, initially framed as a data collection principle, re-emerges at the algorithmic level as a quantifiable fairness constraint on the Scoring Engine. The simulation phase surfaces two systemic risks — the maintenance avalanche and the server scaling ceiling — that static design analysis could not have revealed, underscoring the value of computational validation as a final step before pilot deployment. Addressing the identified limitations, particularly through empirical degradation data and extended multi-semester modelling, will further strengthen the platform's evidence base ahead of full institutional rollout.

Chapter 6

Conclusions and Future Work

6.1 Conclusions

This project set out to design, analyse, and computationally validate a centralised Equipment and Connectivity Management (ECM) platform capable of managing the lifecycle of institutional technological assets for a user base of 7,000 students and faculty at Universidad Distrital Francisco José de Caldas. Across four structured workshops, the project progressed from requirements gathering and architectural design, through robust engineering and risk management, to empirical validation via Discrete-Event Simulation and Agent-Based Modelling. The following conclusions summarise the key achievements of the complete development cycle.

- **Validation of Institutional Need:** The data collection phase confirmed that manual equipment management is the primary operational bottleneck, with 60% of surveyed users rating the current service as regular or below, and 80% having received a device in poor condition at least once. This validates the necessity of a centralised digital platform to ensure equitable and transparent access to technology.
- **Structural Viability of the Model:** The application of Systems Analysis and Design methodologies, Domain-Driven Design, and Clean Architecture principles has produced a multi-tier architecture that handles high-concurrency requests while maintaining data integrity. The three-layer structure — Presentation, Application Logic, and Data & Integration — decouples business rules from technical implementation, ensuring long-term maintainability and fault tolerance.
- **Effectiveness of the Design Decisions:** The conceptual integration of the *Strategy* and *Observer* patterns addresses user requirements for flexible lending policies and real-time availability notifications. The Automated Scoring Engine, validated by simulation, reduces average wait time by 34% compared to a first-come, first-served model, while the Academic Hold mechanism raises the device recovery rate from 85% to 98.7% when activated.
- **Ethical and Methodological Rigour:** The establishment of a strict ethical framework, a Domain-Driven Design approach, and a formal ISO 31000 risk management plan ensures that the system is technically sound, aligned with institutional regulations, and equitable in its resource allocation. The simulation parameter sweep further quantifies this equity commitment, identifying $V = 0.5$ as the optimal vulnerability weight and establishing $V = 0.35$ as the minimum threshold below which allocation equity degrades measurably.

- **Empirical Validation and Risk Discovery:** Computational simulation confirmed four of five core design decisions and surfaced two previously uncharacterised systemic risks — the maintenance avalanche effect and the server scaling ceiling above 1,200 concurrent requests. These findings provide the engineering team with a concrete, evidence-based agenda for the pilot deployment phase, transforming the project from a conceptual blueprint into an empirically grounded implementation plan.

In conclusion, the four phases of this project have successfully translated a high-level administrative problem into a robust, ethically grounded, and computationally validated platform. The ECM system, as designed and validated, represents a feasible and equitable solution to the digital divide challenge identified at the outset, and is ready to proceed to pilot deployment.

6.2 Future Work

The simulation and analysis phases have identified several concrete directions for future development that extend naturally from the work completed.

- **Pilot Deployment and Empirical Calibration:** The immediate next step is a controlled pilot with a subset of institutional technical staff. Once operational, real maintenance logs should replace the synthetic device degradation function used in the ABM, and real request data should be used to recalibrate the Scoring Engine weights for the first quarterly audit cycle.
- **Horizontal Auto-Scaling Implementation:** The stress scenario identified response time degradation above 1,200 concurrent requests. Implementing horizontal auto-scaling triggers at 800 requests per minute is a prerequisite for sustaining the 99.9% uptime SLA during examination periods, and must be addressed before university-wide rollout.
- **Multi-Semester ABM Extension:** The current simulation covers a single 180-day horizon. Extending the ABM to model multi-semester trajectories would allow investigation of how compliance scores evolve over time and whether the Academic Hold mechanism produces lasting behavioural change in returning students.
- **Staggered Maintenance Scheduling Policy:** The maintenance avalanche effect — in which more than 25% of the fleet can enter repair simultaneously — requires a formal scheduling policy that caps concurrent repairs at 10% of the fleet. Implementing and testing this policy within the pilot deployment will validate the mitigation before full-scale operation.
- **Donor Contribution Dynamics:** Inventory growth through donor contributions is currently treated as a fixed initial condition. Modelling donation events as stochastic processes would provide a more realistic picture of long-term fleet evolution and allow the platform to simulate the equity impact of different donor engagement strategies.
- **System Usability Evaluation:** The student portal has been designed to achieve a System Usability Scale (SUS) score above 80. A formal usability study with a representative pilot group will validate this target and identify interface improvements before the university-wide release.

Chapter 7

Reflection

Undertaking the ECM project across four structured workshops has been a formative experience that extended well beyond the acquisition of technical skills. This chapter reflects on the most significant learning outcomes, the decisions that shaped the project's direction, the challenges encountered, and what the team would approach differently given the opportunity.

7.1 From Static Models to Empirical Thinking.

Perhaps the most significant shift in perspective came during the transition from Workshop 3 to Workshop 4. Up to that point, the design process had been primarily analytical: requirements were gathered, architectures were drawn, and risks were catalogued. The introduction of Discrete-Event Simulation and Agent-Based Modelling fundamentally changed how the team reasoned about the system. The maintenance avalanche effect — a failure mode that emerged only under computational simulation and had not appeared in any static risk analysis — demonstrated that complex systems can harbour dynamics that are invisible to structured inspection alone. This lesson, that emergent behaviour requires a different class of analytical tool, is perhaps the most transferable insight from the entire project.

7.2 The Value of Methodological Layering.

The decision to combine three methodological pillars — the SDLC for process structure, Domain-Driven Design for architectural grounding, and Clean Architecture for implementation separation — initially felt overly formal for an academic project. In practice, however, each layer earned its place. The SDLC prevented scope creep between workshops by anchoring each phase to concrete deliverables. DDD kept the team focused on institutional lending as the core domain rather than drifting toward technically interesting but operationally irrelevant features. Clean Architecture made the simulation phase significantly easier by ensuring that business logic could be isolated and tested independently of interface and persistence concerns. In hindsight, the overhead of maintaining this methodological discipline was justified by the coherence it produced across four phases of work.

7.3 Challenges Faced.

The primary technical challenge was calibrating the Scoring Engine formula without access to longitudinal institutional data. The vulnerability, academic load, and compliance weights were derived from a single-point-in-time survey, which introduced uncertainty into the equity

analysis. The team mitigated this through the parameter sweep conducted in Workshop 4, but a true calibration would require at minimum one full semester of operational data. A secondary challenge was the gap between the simulated system and the real institutional environment: the ABM's device degradation function, donor contribution model, and student compliance distributions are all approximations. Managing the tension between model fidelity and computational tractability required ongoing judgement calls that sometimes felt underdetermined.

7.4 What Would Be Done Differently.

Were the project to begin again, the team would invest earlier in defining quantitative acceptance criteria for the equity dimension of the Scoring Engine. The fairness constraint — that V must remain above 0.35 — was discovered through simulation in the final workshop rather than specified as a design requirement from the outset. Treating algorithmic fairness as a first-class, measurable requirement from Workshop 1 would have produced a more defensible and auditable design from the beginning. Additionally, the survey instrument would be redesigned to include longitudinal follow-up and complementary institutional data, reducing the self-selection bias identified in the Discussion chapter.

7.5 Looking Forward.

The project has reinforced a conviction that the boundary between systems engineering and social responsibility is not a boundary at all. Every design decision in the ECM platform — the scoring weights, the Academic Hold threshold, the maintenance scheduling policy — carries a direct consequence for a student's access to learning resources. Approaching those decisions with both technical rigour and ethical intentionality has been the most meaningful aspect of this work, and one that will inform future engineering practice beyond the scope of this project.

References

Banks, A. and Porcello, E. (2023), *Learning React: Modern Patterns for Developing React Apps*, 2nd edn, O'Reilly Media, Sebastopol, CA.

Banks, J., Carson, J. S., Nelson, B. L. and Nicol, D. M. (2010), *Discrete-Event System Simulation*, 5th edn, Pearson, Upper Saddle River, NJ.

Bass, L., Clements, P. and Kazman, R. (2021), *Software Architecture in Practice*, 4th edn, Addison-Wesley, Boston, MA.

Brooke, J. (1996), 'Sus: A quick and dirty usability scale', *Usability evaluation in industry* .

Evans, E. (2003), *Domain-Driven Design: Tackling Complexity in the Heart of Software*, Addison-Wesley Professional, Boston, MA, USA.

Fowler, M., Rice, D., Foemmel, M., Hieatt, E., Mee, R. and Stafford, R. (2002), *Patterns of Enterprise Application Architecture*, Addison-Wesley Professional, Boston, MA, USA.
URL: *https://sar.ac.id/stmik$_e$book/prog$_f$ile$_f$ile/EFCofwzsj0.pdf*

International Organization for Standardization (2011), 'Systems and software engineering — systems and software quality requirements and evaluation (SQuaRE) — system and software quality models'.

International Organization for Standardization (2018), 'Risk management — guidelines'.

Kottwitz, S. (2021), *LaTeX beginner's guide : create visually appealing texts, articles, and books for business and science using LaTeX*, Packt. ISBN: 9781801072588.

Lamport, L. (1994), *LATEX: a document preparation system: user's guide and reference manual*, Addison-wesley.

Macal, C. M. and North, M. J. (2010), 'Tutorial on agent-based modelling and simulation', *Journal of Simulation* **4**(3), 151–162.

Martin, R. C. (2017), *Clean Architecture: A Craftsman's Guide to Software Structure and Design*, Prentice Hall, Upper Saddle River, NJ.

Patriarca, R., Bergström, J., Di Gravio, G. and Costantino, F. (2018), 'Resilience engineering: Current status of the research and future challenges', *Safety Science* **102**, 79–100.
URL: *https://www.sciencedirect.com/science/article/pii/S0925753516306130*

Project Management Institute (2021), *A Guide to the Project Management Body of Knowledge (PMBOK Guide)*, 7th edn, Project Management Institute, Newtown Square, PA.

Shelly, G. B., Rosenblatt, H. J. and Cashman, T. J. (2011), *Systems Analysis and Design*, 9th edn, Course Technology Ptr, Boston, MA, USA.

Sommerville, I. (2015), *Software Engineering*, 10th edn, Pearson.